

Agentic QA Engine

AI-Powered Test Case Generation Platform

Senior Leadership Demo

Trade Processing Platform · June 2026

The Problem We Solved

Manual QA test generation is a **bottleneck** at every release cycle.

Pain Point	Impact
QA engineers write test cases by hand	3–5 days per feature module
Requirements scattered across Confluence	Context lost between teams
No traceability from requirement → test	Audit failures, missed edge cases
Playwright scripts written from scratch	Inconsistent coverage

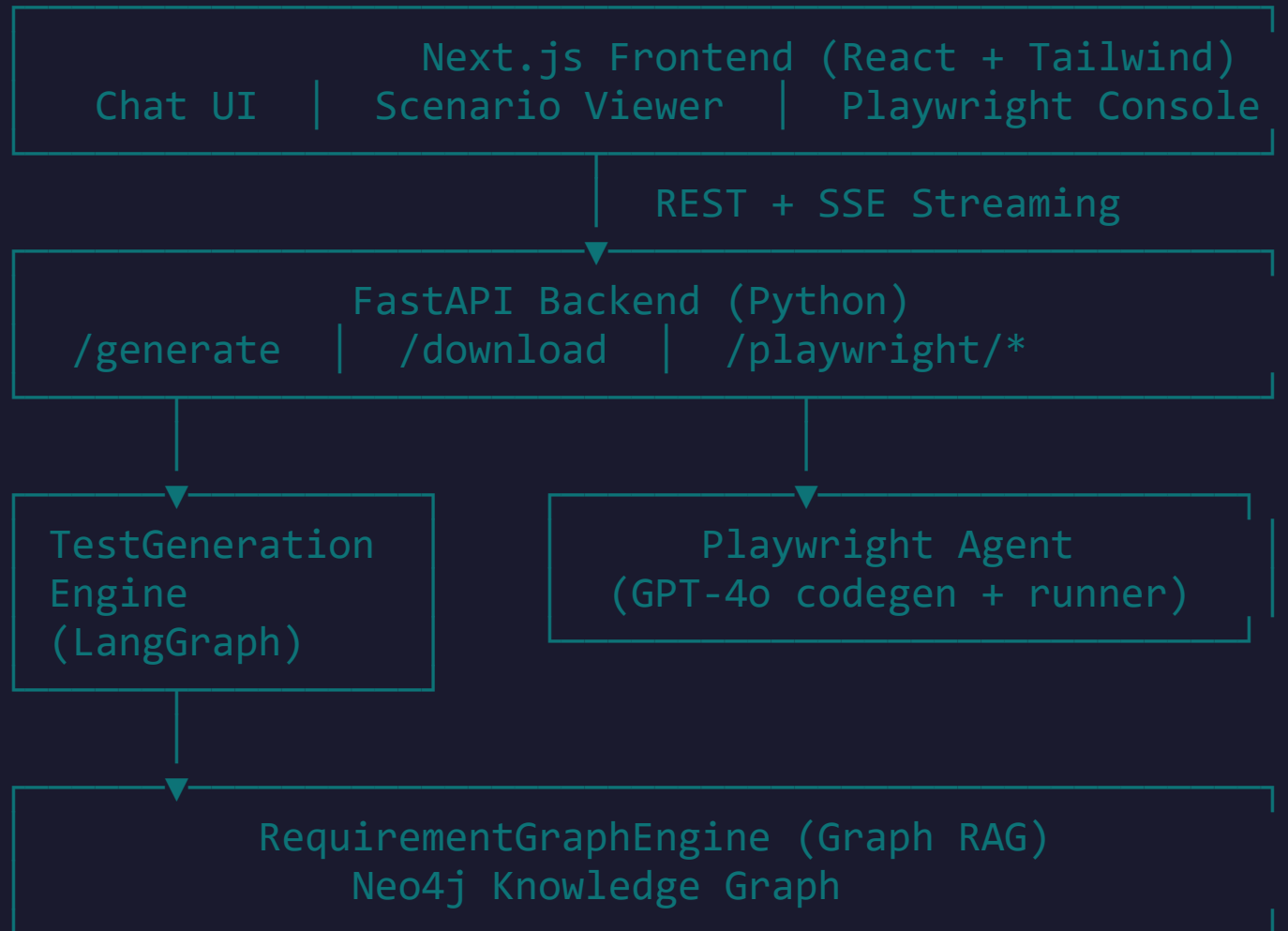
Goal: Turn a natural-language question into executable BDD test cases — in seconds, not days.

Solution at a Glance

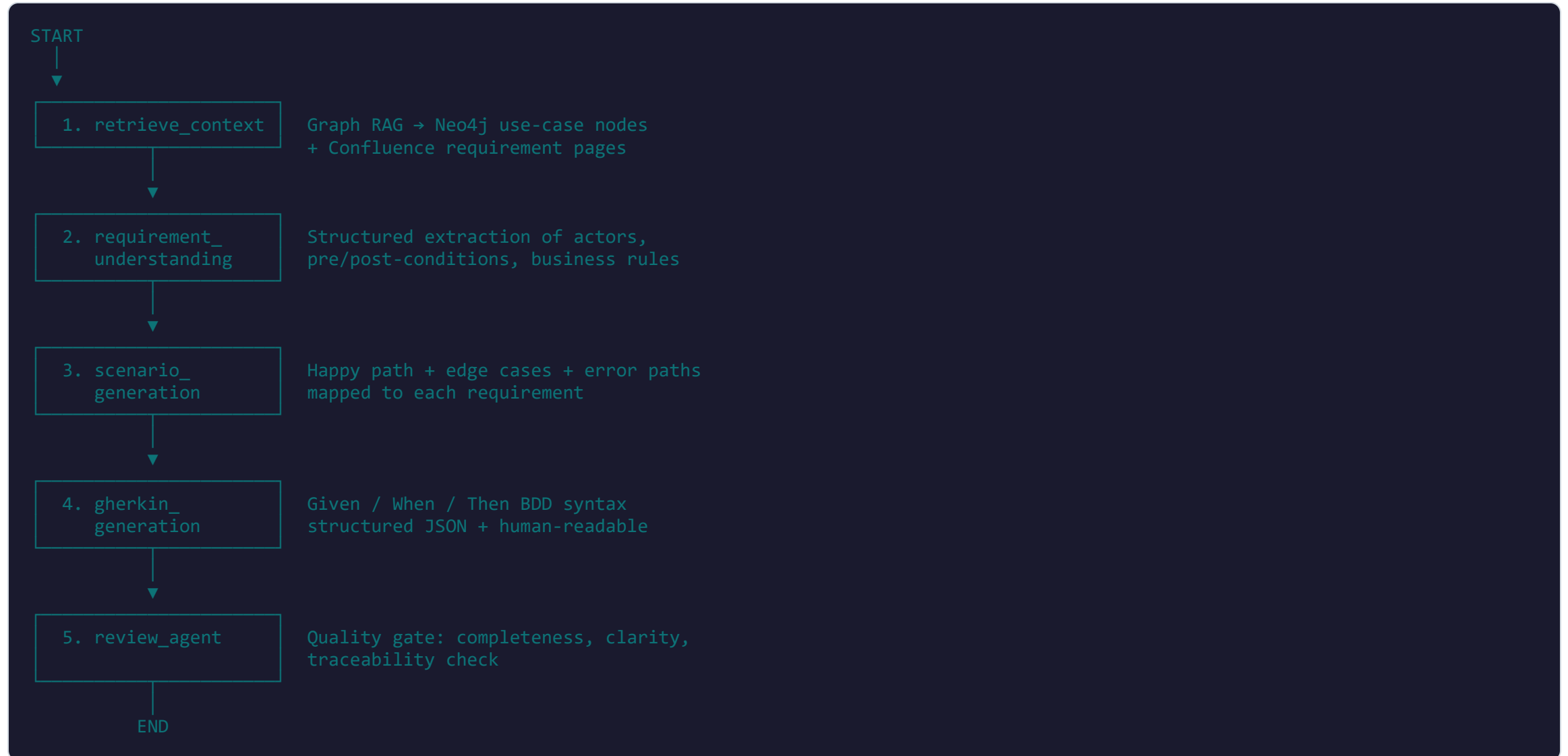
```
graph TD; A[Engineer types a question] --> B[AI Pipeline (LangGraph)]; B --> C[Gherkin Test Cases ↔ Human Review]; C --> D[Playwright Scripts (auto-generated)]; D --> E[Browser Test Execution + Failure Triage];
```

- No prompt engineering expertise required from QA engineers
- Full traceability — every scenario linked to a source requirement
- One-click from question to running browser tests

Full-Stack Architecture



LangGraph Pipeline — 5 Nodes



Knowledge Layer — Graph RAG

RequirementGraphEngine wraps Neo4j with semantic retrieval.

```
# What happens at Node 1
context = engine.retrieve_relevant_context(question)
# → vector similarity over use-case embeddings
# → graph traversal: use_case → requirements → actors → error_codes
```

Why a Knowledge Graph?

- Requirements have **relationships** — a graph captures what flat docs cannot
- `get_use_case_graph` returns the full subgraph for any module
- `debug_rag_retrieval` exposes what the model actually sees

Confluence Integration

- `list_requirement_pages` → enumerate all BA requirement docs
- `get_all_requirements` → bulk load into the graph on rebuild

MCP Servers — Claude Code Integration

Two FastMCP servers expose the knowledge base as tools directly in Claude Code.

Server	Tools	Purpose
<code>neo4j_server.py</code>	<code>run_cypher</code> , <code>get_schema</code> , <code>get_use_case_graph</code> , <code>list_all_use_cases</code> , <code>get_error_codes</code> , <code>debug_rag_retrieval</code>	Query the knowledge graph live
<code>confluence_server.py</code>	<code>get_page</code> , <code>list_requirement_pages</code> , <code>search_pages</code> , <code>get_all_requirements</code> , <code>create_test_plan</code>	Read & write Confluence docs

Engineers can ask Claude Code: *"Show me all error codes for the Trade Settlement use case"* — answered directly from Neo4j, no manual query.

Playwright Agent — End-to-End Automation

Once Gherkin scenarios are approved, the **Playwright Agent** takes over.

Approved Gherkin Suite



GPT-4o Codegen

- └ Page Object files (tests/pages/)
- └ Test scripts (tests/test_suites/)



pytest-playwright runner

- └ Chromium (headless)
- └ Failure artifacts: .png screenshots + .html DOM snapshots



AI Failure Triage Agent

- └ Classifies each failure (selector / timing / logic / env)
- └ Suggests targeted fix → one-click Apply

Playwright Agent — Key Capabilities

Capability	Description
<code>generate_suite_preview</code>	Dry-run: generate scripts without saving
<code>generate_and_run_suite</code>	Full cycle: generate → save → execute
<code>trriage_failures_agent</code>	AI analysis of failed tests with fix suggestions
<code>apply_test_fix</code>	Patches the script file with the suggested fix
<code>run_single_test</code>	Re-run one test after a fix to confirm it passes
<code>regenerate_scripts</code>	Re-generate scripts from updated Gherkin

Models used:

- `gpt-4o` — Playwright script generation (high accuracy code)
- `gpt-4o-mini` — Failure triage & review (fast, cost-efficient)

Frontend — QA Engineer Workflow

1. Chat Input
"Generate test cases for User Role Management" |
↓ streaming response
2. Scenario Viewer
Tabbed view: Requirements | Scenarios | Gherkin
↓ approve
3. Playwright Console
Generate Scripts → Run Suite → View Results
↓
4. Triage Panel
Failure list → AI analysis → Apply Fix button

Built with **Next.js 14** · **TypeScript** · **Tailwind CSS**

Server-Sent Events (SSE) for real-time pipeline streaming

Technology Stack

Layer	Technology	Role
Frontend	Next.js 14, TypeScript, Tailwind	QA engineer UI
Backend	FastAPI, Python	REST API + SSE streaming
AI Pipeline	LangGraph, LangChain	Multi-agent orchestration
LLM	GPT-4o, GPT-4o-mini	Generation + review
Knowledge Graph	Neo4j + Graph RAG	Requirement retrieval
Docs	Confluence (MCP)	Source of truth
Test Execution	Playwright + pytest	Browser automation
MCP Servers	FastMCP	Claude Code integration

What It Delivers

Time Savings

- **Manual:** 3–5 days to write test cases for one module
- **With Agentic QA Engine:** Under 2 minutes end-to-end

Coverage

- Happy path + edge cases + error paths — generated automatically
- Every scenario traceable back to a Confluence requirement page

Quality Gate

- `review_agent` enforces completeness, clarity, and BDD correctness before any script is generated
- AI triage catches root cause of test failures — not just "test failed"

Live Demo Flow

1. Enter a question in the chat UI

"Create test cases for User and Role Management in the Admin module"

2. Watch the pipeline stream — 5 nodes execute in sequence

3. Review Gherkin output — structured scenarios with Given/When/Then

4. Approve & generate Playwright scripts — one click

5. Run the suite — browser tests execute against the application

6. View triage results — failures analyzed with fix suggestions

7. Apply fix & rerun — confirm the test passes

Roadmap

Priority	Item	Status
Done	LangGraph 5-node pipeline	✓
Done	FastAPI + Next.js full-stack	✓
Done	Playwright script gen + execution	✓
Done	AI failure triage with fix	✓
Done	Neo4j + Confluence MCP servers	✓
Next	Dependency agent (wired into pipeline)	🔄
Next	Playwright MCP server for Claude Code	🔄
Next	GitHub MCP — auto-PR for approved suites	🔄
Future	Multi-browser parallel execution	📋
Future	Historical test trend dashboard	📋

Thank You

Questions?

Agentic QA Engine · Trade Processing Platform

uppalayk105@gmail.com